Recent Advances in Neural Network Methods for Solving Partial Differential Equations

A Comprehensive Review

1 Introduction

Partial differential equations (PDEs) are fundamental tools for modeling complex physical phenomena in various scientific and engineering disciplines. Traditional numerical methods for solving PDEs, such as finite difference methods (FDMs), finite element methods (FEMs), and spectral methods, have dominated the field for decades. However, these methods often face challenges when dealing with complex geometries, high-dimensional problems, and multiscale phenomena.

In recent years, neural network-based approaches have emerged as promising alternatives for solving PDEs. These methods offer several advantages, including mesh-free approximation, automatic differentiation capabilities, and natural handling of complex geometries. This survey paper reviews recent developments in neural network methods for solving PDEs, with a focus on randomized methods, linearized approaches, and methods designed for specific PDE types and problem dimensions.

2 Foundational Work on Randomized Neural Networks

Before examining specific PDE solvers, it is essential to understand the foundational theories underpinning randomized neural networks, which serve as the basis for many modern approaches.

2.1 Random Vector Functional-Link Networks

The seminal work of Igelnik and Pao [Igelnik and Pao, 1995] on Random Vector Functional-Link (RVFL) networks established much of the theoretical

groundwork for randomized neural networks. Their approach used a limit-integral representation of the function to be approximated and evaluated the integral through the Monte-Carlo approach. The RVFL neural network is defined as:

$$f_{w_n}(x) = \sum_{k=1}^{n} a_k g(w_k \cdot x + b_k) \tag{1}$$

where a_k are trainable output layer weights while $w_k \in \mathbb{R}^d$ and $b_k \in \mathbb{R}$ are randomly assigned and fixed hidden layer parameters.

The key theoretical findings from this work include:

1. RVFL networks are universal approximators for continuous functions on any compact subset of $\mathbb{R}^d$.

2. For Lipschitz continuous functions, the approximation error converges at a rate of $O(C/\sqrt{n})$ with C independent of the dimension d, effectively overcoming the curse of dimensionality compared to fixed basis methods where the error decays as $O(n^{-1/d})$.

3. Similar convergence properties hold for randomized radial basis function networks.

This work established that randomizing inner layer parameters while only training output weights could still yield universal approximation capabilities with faster learning and better scaling to high dimensions.

2.2 Extreme Learning Machines: Theory and Initial Applications

Building on these randomization principles, Huang et al. [Huang et al., 2006] introduced the Extreme Learning Machine (ELM) framework for single-hidden layer feedforward neural networks (SLFNs). Their approach directly addresses the primary bottleneck in neural networks: slow learning speed due to gradient-based optimization of all network parameters.

In ELM, the weights connecting inputs to hidden nodes are randomly assigned and fixed, while only the weights connecting hidden nodes to outputs are optimized. For N arbitrary distinct samples (x_i, t_i), standard SLFNs with $\tilde{N}$ hidden nodes and activation function $g(x)$ can be mathematically modeled as:

$$\sum_{i=1}^{\tilde{N}} \beta_i g(w_i \cdot x_j + b_i) = o_j, \quad j = 1, \ldots, N \tag{2}$$

which can be written compactly as $H\beta = T$, where H is the hidden layer output matrix. The ELM approach recognizes that training reduces to finding the minimum norm least-squares solution $\hat{\beta} = H^{\dagger}T$, where $H^{\dagger}$ is the Moore-Penrose generalized inverse.

The key theoretical contributions include:

1. Proving that with randomly assigned input weights and biases, SLFNs can learn N distinct samples with zero error using at most N hidden nodes, given that the activation function is infinitely differentiable.

2. Demonstrating that SLFNs with randomly assigned parameters can universally approximate any continuous function and its derivatives.

3. Showing that the minimum norm solution found by ELM tends to have better generalization performance according to Bartlett's theory on feedforward neural networks.

Experimental results across various benchmarks demonstrated that ELM could achieve not only much faster learning (up to thousands of times faster than backpropagation and SVMs) but also superior generalization performance in many cases.

2.3 Theoretical Assessment of ELM Feasibility

While early work established ELM's practical effectiveness, Liu et al. [Liu et al., 2015] provided a comprehensive theoretical assessment of ELM's feasibility, addressing fundamental questions about generalization capabilities and conditions for stable performance.

For functions with smoothness order r, they established that properly configured ELM can achieve the optimal generalization bound of $O(m^{-2r/(2r+d)})$ with high probability, where m is the number of samples and d is the input dimension. This matches the theoretical limit for traditional fully-trained neural networks.

The analysis revealed several important insights:

1. Different activation functions (polynomial, Nadaraya-Watson, sigmoid) all maintain optimal generalization capability when properly configured, with the number of hidden neurons optimally set as $n = [m^{d/(d+2r)}]$ for polynomial activations.

2. For guaranteed stable performance, either using polynomial activation functions or employing Tikhonov regularization effectively ensures the weak regularity of the hidden layer output matrix.

3. The regularized ELM estimator defined by $\alpha^* = (H^T H + \lambda m I)^{-1} H^T y$ maintains optimal generalization bounds with appropriate choices of the regularization parameter λ.

This theoretical assessment provided the first rigorous explanation for why randomly assigned inner layer parameters work so well in practice, establishing that ELM's computational efficiency comes with no inherent cost in generalization capability when properly implemented.

These foundational works established the theoretical underpinnings for the more specialized randomized neural network methods that have recently been developed for solving PDEs, which we examine in the following sections.

3 Theoretical Foundations of Linearized Neural Networks

Recent theoretical work by Liu, Mao, and Xu [Liu et al., 2025] has provided rigorous mathematical foundations for linearized neural networks with ReLU^k activation functions, offering profound insights into why randomized neural network methods work and establishing tight error bounds for their performance. This theoretical framework provides a deep understanding of the approximation capabilities of linearized neural networks, particularly when applied to solving PDEs.

3.1 Function Spaces and Neural Network Representations

The theoretical analysis focuses on shallow neural networks with ReLU^k activation function σ_k, defined as:

$$\Sigma_n^k = \left\{ \sum_{j=1}^{n} a_j \sigma_k(\theta_j \cdot \tilde{x}) : \theta_j \in \mathbb{S}^d, a_j \in \mathbb{R} \right\} \tag{3}$$

where $\tilde{x} = \begin{pmatrix} x \\ 1 \end{pmatrix}$ and $\theta_j = \begin{pmatrix} w_j \\ b_j \end{pmatrix}$. The constrained version with bounded parameters is given by:

$$\Sigma_{n,M}^k = \left\{ \sum_{j=1}^n a_j \sigma_k(\theta_j \cdot \tilde{x}) : \theta_j \in \mathbb{S}^d, \sum_{j=1}^n |a_j| \leq M \right\} \tag{4}$$

The key innovation in [Liu et al., 2025] is the introduction of the finite neuron space (FNS), a linear subspace of Σ_n^k where the parameters $\{\theta_j^*\}_{j=1}^n$ are fixed:

$$L_n^k = L_n^k(\{\theta_j^*\}_{j=1}^n) = \text{span}\{\phi_1, \ldots, \phi_n\} \tag{5}$$

where $\phi_j(x) = \sigma_k(\theta_j^* \cdot \tilde{x})$. The constrained FNS is defined as:

$$L_{n,M}^k = \left\{ \sum_{j=1}^n a_j \phi_j : \left(\sum_{j=1}^n a_j^2 \right)^{1/2} \leq M \right\} \tag{6}$$

This formulation is particularly relevant to ELM-based methods for PDEs, as it provides a rigorous mathematical foundation for linearized approaches where only the output weights are trained.

3.2 Integral Representation of Sobolev Spaces

A groundbreaking contribution of [Liu et al., 2025] is the novel integral representation of Sobolev spaces using ReLU^k functions. The authors prove that the Sobolev space $H^{d+2k+1/2}(\Omega)$ can be characterized as:

$$H^{d+2k+1/2}(\Omega) = \left\{ \int_{\mathbb{S}^d} \sigma_k(\theta \cdot \tilde{x}) \psi(\theta) d\theta : \psi \in L^2(\mathbb{S}^d) \right\} \tag{7}$$

This representation establishes that for any function $f \in H^{d+2k+1/2}(\Omega)$, there exists $\psi \in L^2(\mathbb{S}^d)$ such that:

$$f(x) = \int_{\mathbb{S}^d} \sigma_k(\theta \cdot \tilde{x}) \psi(\theta) d\theta \tag{8}$$

with the norm equivalence:

$$\|f\|_{H^{d+2k+1/2}(\Omega)} \simeq \inf_{\psi \in L^2(\mathbb{S}^d)} \{ \|\psi\|_{L^2(\mathbb{S}^d)} : f(x) = \int_{\mathbb{S}^d} \sigma_k(\theta \cdot \tilde{x}) \psi(\theta) d\theta \} \tag{9}$$

This result complements the integral representation of Barron spaces and establishes a connection between Sobolev regularity and neural network approximation. It demonstrates that functions in the Sobolev space $H^{d+2k+1/2}(\Omega)$ can be naturally represented using ReLU^k basis functions, providing a deeper understanding of which functions are effectively approximable by neural networks.

3.3 Optimal Approximation Rates for Linearized Networks

Another crucial finding is that linearized networks with well-distributed parameters can achieve optimal approximation rates. For well-distributed parameters $\{\theta_j^*\}_{j=1}^n \subset \mathbb{S}^d$ (where the mesh size $h = \max_{\theta\in\mathbb{S}^d} \min_{1\le j\le n} \rho(\theta, \theta_j^*)$ satisfies $h \lesssim n^{-1/d}$) and any $f \in H^{d+2k+1/2}(\Omega)$, with $M \simeq \|f\|_{H^{d+2k+1/2}(\Omega)}$:

$$\inf_{f_n \in L^k_{n,M}} \|f - f_n\|_{L^2(\Omega)} \lesssim n^{-1/2-2k+1/2d} \|f\|_{H^{d+2k+1/2}(\Omega)} \tag{10}$$

This rate matches the optimal approximation rate for nonlinear shallow networks (up to logarithmic factors), suggesting that linearized networks are as powerful as their nonlinear counterparts for functions in appropriate Sobolev spaces. This theoretical result confirms the empirical success of ELM-based methods for PDEs, explaining why fixing inner layer parameters and only training output weights can still achieve excellent approximation capabilities.

3.4 Coefficient Bound Estimates

The tight coefficient estimate derived in [Liu et al., 2025]:

$$\sqrt{n}\|a\|_2 \lesssim \|f\|_{H^{d+2k+1/2}(\Omega)} \tag{11}$$

provides crucial insights for the numerical stability and generalization capabilities of linearized neural networks. This bound ensures that the coefficients in the network expansion remain controlled, which is essential for both numerical stability and generalization analysis in machine learning applications.

3.5 Implications for Randomized Neural Network Methods

These theoretical findings have profound implications for the randomized neural network methods discussed in this survey. They establish that the

success of methods like PIELM, ELM-FBPINN, and locELM is not coincidental but grounded in solid mathematical theory. Several key insights emerge:

1. **Well-distributed vs. Random Parameters**: The theory suggests that the effectiveness of randomized approaches stems primarily from the fact that randomly generated weights tend to be well-distributed with high probability, rather than any intrinsic advantage of randomness. This explains the empirical observation that randomized neural networks for PDEs can achieve high accuracy despite fixing inner layer parameters.

2. **Linear vs. Nonlinear Training**: The optimal approximation rates achievable by linearized networks indicate that the computationally intensive nonlinear training of all network parameters may be unnecessary for certain problem classes, supporting the efficiency advantages of ELM-based methods observed in practice.

3. **Deterministic Parameter Selection**: The theory suggests that deterministically chosen well-distributed parameters could potentially outperform random parameter selection, pointing to a promising direction for further improving the already efficient ELM-based methods.

4. **Error Bounds for Sobolev Functions**: The established error bounds provide theoretical guarantees for the performance of linearized neural networks on functions with Sobolev regularity, which includes solutions to many PDEs of practical interest.

These theoretical results provide a rigorous explanation for the effectiveness of the various randomized neural network methods for PDEs discussed throughout this survey. Methods like PIELM, locELM, and RRNN can now be understood not just as empirically successful approaches but as implementations that leverage the optimal approximation properties of linearized networks with well-distributed parameters.

4 Physics-Informed Neural Networks and Their Limitations

Physics-Informed Neural Networks (PINNs) represent a significant advancement in scientific machine learning. They incorporate physical laws as con-

straints into the neural network training process, enabling the solution of PDEs without labeled data.

The general framework for PINNs involves:

1. Defining a boundary value problem of the form:

$$\mathcal{N}[u(x)] = f(x), \quad x \in \Omega \subset \mathbb{R}^d, \tag{12}$$

$$\mathcal{B}_k[u(x)] = g_k(x), \quad x \in \Gamma_k \subset \partial\Omega, \tag{13}$$

   where $\mathcal{N}[u(x)]$ is a differential operator and $\{\mathcal{B}_k[\cdot]\}$ represents boundary conditions.

2. Parameterizing the solution using a neural network $u(x, \boldsymbol{\theta})$ with parameters $\boldsymbol{\theta}$.

3. Training the network by minimizing a composite loss function:

$$\mathcal{L}(\boldsymbol{\theta}) = \lambda_I \sum_{i=1}^{N_I} \|\mathcal{N}[u(x_i, \boldsymbol{\theta})] - f(x_i)\|^2 + \sum_{k=1}^{K} \lambda_B^{(k)} \sum_{i=1}^{N_B^{(k)}} \|\mathcal{B}_k[u(x_i^{(k)}, \boldsymbol{\theta})] - g_k(x_i^{(k)})\|^2 \tag{14}$$

Despite their flexibility, PINNs face several challenges:

- **Spectral Bias**: PINNs struggle to capture high-frequency components of solutions, favoring smoother, low-frequency components.
- **Training Efficiency**: The non-convex optimization problem requires gradient-based methods, leading to slow convergence and high computational costs.
- **Parameter Sensitivity**: Performance can be highly dependent on network architecture, initialization, and hyperparameter choices.
- **Multiscale Problems**: Traditional PINNs often fail to accurately represent solutions with features at multiple scales.

Recent research has focused on addressing these limitations through various innovations.

5 Physics-Informed Extreme Learning Machine Methods

5.1 Original PIELM Formulation

Building on the foundational ELM work, the Physics-Informed Extreme Learning Machine (PIELM) [Dwivedi and Srinivasan, 2020a] emerged as a significant advancement for solving PDEs through neural network approaches. PIELM combines the physics-informed loss function approach with the computational efficiency of ELM, addressing several key limitations of traditional PINNs.

The PIELM method employs a single-layer feedforward neural network with randomly assigned input layer weights that remain fixed (non-trainable). For a PIELM with N^* neurons in the hidden layer, the output is calculated as:

$$f(\vec{\chi}) = \vec{h}\vec{c} \tag{15}$$

where $\vec{h}$ represents the outputs of the hidden neurons and $\vec{c}$ contains the output layer weights. The key innovation lies in incorporating the physics of PDEs into the training process. For a PDE of the form:

$$\frac{\partial}{\partial t}u(x,t) + \mathcal{N}u(x,t) = R(x,t), \quad (x,t) \in \Omega \tag{16}$$

$$u(x,t) = B(x,t), \quad (x,t) \in \partial\Omega \tag{17}$$

$$u(x,0) = F(x), \quad x \in (x_L, x_R) \tag{18}$$

The errors in approximating the PDE, boundary conditions, and initial conditions are defined and combined into a system of linear equations:

$$H\vec{c} = \vec{K} \tag{19}$$

Unlike traditional PINNs that require gradient-based optimization, PIELM directly solves this system using the Moore-Penrose generalized inverse:

$$\vec{c} = \text{pinv}(H)\vec{K} \tag{20}$$

This analytical determination of output weights eliminates the need for iterative optimization, resulting in training times that are orders of magnitude faster than gradient-based approaches. A significant advantage of PIELM is its systematic approach to architecture selection - setting the number of hidden units equal to the number of constraints ($N^* = N_f + N_{bc} + N_{ic}$) typically yields optimal results.

Numerical experiments on various linear PDEs, including advection-diffusion equations in complex domains, demonstrated PIELM's efficiency, achieving accuracies of 10^{-4} to 10^{-6} in just seconds, compared to hours required by traditional PINNs for similar problems.

For PDEs with sharp gradient solutions, which pose challenges for standard PIELM, a distributed version (DPIELM) was developed that divides the computational domain into multiple sub-domains, with each sub-domain represented by a separate PIELM. This approach successfully handled challenging cases including advection of high-frequency wave packets that deep PINNs failed to capture.

5.2 PIELM for Biharmonic Equations

The extension of PIELM to biharmonic equations [Dwivedi and Srinivasan, 2020b] represents an important application to high-order PDEs, which are central to modeling numerous physical phenomena including viscous fluid flows, geophysical flows, and structural mechanics. The biharmonic equation in two dimensions takes the form:

$$\nabla^4 u(x,y) = \frac{\partial^4 u}{\partial x^4} + 2\frac{\partial^4 u}{\partial x^2 \partial y^2} + \frac{\partial^4 u}{\partial y^4} = R(x,y), \quad (x,y) \in \Omega \tag{21}$$

with appropriate boundary conditions. The PIELM approach incorporates physics-based training errors corresponding to the PDE residual and boundary condition violations:

$$\xi_R = \frac{\partial^4 f}{\partial x^4} + 2\frac{\partial^4 f}{\partial x^2 \partial y^2} + \frac{\partial^4 f}{\partial y^4} - R, \quad (x,y) \in \Omega \tag{22}$$

$$\xi_{G1} = f - G_1, \quad (x,y) \in \partial\Omega \tag{23}$$

$$\xi_{G2} = \frac{\partial f}{\partial n} - G_2, \quad (x,y) \in \partial\Omega \tag{24}$$

These constraints lead to a system of linear equations that can be solved directly for the output weights.

Numerical experiments on both regular and irregular domains demonstrated PIELM's effectiveness, achieving accuracy of order 10^{-6} with computation times on the order of milliseconds, approximately 1000 times faster than comparable PINNs. A particularly impressive demonstration involved solving the biharmonic equation on a scaled map of Australia - a complex geometry where standard mesh generation tools failed, highlighting PIELM's advantage for problems with complicated geometries.

5.3 Kernel-Based ELM for Elliptic PDEs

The Extreme Learning Machine with kernels (KELM) [Li et al., 2023] introduces another variation for solving elliptic PDEs. Unlike standard PIELM, KELM employs a kernel function to replace the activation function, expressing the solution as:

$$o_j = \begin{bmatrix} K(X_j, X_1) \\ K(X_j, X_2) \\ \vdots \\ K(X_j, X_N) \end{bmatrix}^T (\Omega + \lambda I)^{-1} T \tag{25}$$

where $K(X_i, X_j)$ is typically a Gaussian kernel function. The KELM approach combines this with a trial solution construction that automatically satisfies boundary conditions:

$$z(x, y) = A(x, y) + B(x, y)u(x, y) \tag{26}$$

where $A(x, y)$ incorporates boundary information and $B(x, y)$ ensures boundary conditions are satisfied, with $u(x, y)$ being the KELM output.

Numerical examples including various elliptic PDEs demonstrated KELM's superior performance compared to other methods such as Chebyshev neural networks and wavelet neural networks, achieving absolute errors less than 10^{-6} with relatively few training points. A key advantage of KELM over standard PIELM is that it doesn't require specification of the number of hidden neurons, reducing parameter tuning requirements and improving stability.

5.4 Comparison of ELM-Based Methods for PDEs

ELM-based methods for PDEs share several common advantages over traditional PINNs and mesh-based numerical methods:

1. **Computational Efficiency**: By avoiding gradient-based training, these methods achieve solution times orders of magnitude faster than traditional PINNs.

2. **Systematic Architecture Selection**: Unlike deep neural networks that rely on trial and error, ELM-based methods provide clear criteria for determining network architecture.

3. **Optimization Robustness**: Single-layer networks avoid optimization difficulties like vanishing gradients and local minima.

4. **Complex Geometry Handling**: The meshless nature makes these methods ideal for problems with complicated geometries.

5. **Direct Analytical Solution**: For linear PDEs, these methods provide direct analytical solutions, avoiding iterative optimization.

Despite these advantages, ELM-based methods also face limitations. The original PIELM formulation applies primarily to linear PDEs, as nonlinear operators lead to nonlinear algebraic systems that cannot be solved using simple matrix inversion. Additionally, while these methods excel in handling complex geometries, for regular domains, traditional mesh-based methods often achieve faster computation times.

The evolution of ELM-based methods for PDEs represents an important bridge between the foundational randomized neural network theories and their practical application to challenging PDE problems. These approaches demonstrate how the theoretical advantages of randomization established by early works can be leveraged to create highly efficient solvers for various types of PDEs.

6 Domain Decomposition with Randomized Neural Networks

Building on the success of ELM-based methods, researchers have developed domain decomposition approaches that further enhance performance and accuracy for solving complex PDEs. These methods combine the efficiency of random feature methods with principles from classical domain decomposition to create highly effective PDE solvers.

6.1 Local Extreme Learning Machine (locELM)

The Local Extreme Learning Machine (locELM) method [Dong and Li, 2021] represents a significant advancement in neural network approaches for solving PDEs by combining three key ideas: domain decomposition, local neural networks, and extreme learning machine training. This approach addresses critical limitations of both deep neural networks and standard ELM methods.

In the locELM framework, the domain Ω is partitioned into N_e non-overlapping sub-domains:

$$\Omega = \Omega_1 \cup \Omega_2 \cup \cdots \cup \Omega_{N_e} \tag{27}$$

Each sub-domain Ω_s is assigned a separate shallow feed-forward neural network to represent the local solution:

$$u_i^s(x) = \sum_{j=1}^{M} V_j^s(x) w_{ji}^s, \quad x \in \Omega_s, \quad 1 \leq i \leq N \tag{28}$$

where $V_j^s(x)$ represents the output of the last hidden layer and w_{ji}^s are the trainable output layer weights.

A critical innovation in locELM is the imposition of C^k continuity conditions across sub-domain boundaries, where k depends on the order of the PDE. These continuity conditions are crucial for coupling the local neural networks together, allowing information to propagate between sub-domains while maintaining the required smoothness of the solution.

For linear PDEs, the training process involves:

1. Placing collocation points within each sub-domain and on boundaries
2. Enforcing the PDE on interior collocation points
3. Enforcing boundary conditions on domain boundaries
4. Enforcing continuity conditions across sub-domain boundaries
5. Formulating these constraints as a system of linear equations
6. Solving the resulting system using linear least squares minimization

For nonlinear PDEs, locELM offers two solution approaches:

1. **NLSQ-perturb** (Nonlinear Least Squares with Perturbations): Uses nonlinear least squares optimization with random perturbations to avoid local minima
2. **Newton-LLSQ** (Newton-Linear Least Squares): Linearizes the PDE using Newton's method and solves for the increment field using linear least squares

For time-dependent PDEs, locELM employs a block time-marching strategy, dividing the spatial-temporal domain into successive time blocks and solving each block separately.

Numerical experiments across various linear and nonlinear PDEs demonstrate locELM's exceptional performance:

- Exponential or near-exponential convergence with respect to degrees of freedom
- Accuracy improvements of several orders of magnitude compared to DGM/PINN methods
- Computational efficiency comparable to or better than high-order finite element methods (FEM)
- Error levels reaching 10^{-8} to 10^{-12} for many test problems

The domain decomposition approach in locELM provides additional benefits, including reduced computational cost, sparse coefficient matrices in the least squares problem, and maintained or improved accuracy compared to single-domain approaches.

6.2 Optimal Hyperparameter Computation for ELM Methods

A critical aspect of ELM-based PDE solvers is the selection of hyperparameters, particularly R_m, the maximum magnitude of random coefficients used to initialize the hidden layers. Dong and Yang [Dong and Yang, 2021] addressed this challenge by developing a systematic algorithm for computing optimal R_m values to maximize solution accuracy.

The algorithm identifies two primary ELM configurations:

1. **Single-R_m-ELM**: Uses the same R_m value for all hidden layers
2. **Multi-R_m-ELM**: Employs different R_m values for different hidden layers, represented as $\mathbf{R_m} = (R_m^{(1)}, R_m^{(2)}, ..., R_m^{(L-1)})$

The optimal R_m is computed by minimizing the residual norm of the least squares solution:

$$R_{m0} = \arg\min_{R_m} \|r(R_m)\| \tag{29}$$

or for Multi-R_m-ELM:

$$\mathbf{R_{m0}} = \arg\min_{\mathbf{R_m}} \|r(\mathbf{R_m})\| \tag{30}$$

The optimization employs a differential evolution algorithm which:

1. Generates a population of candidate R_m values

2. Evaluates the cost function for each candidate

3. Updates the population using differential evolution operations

4. Repeats until convergence

This approach is a pre-processing procedure that only needs to be performed once for a given problem, and the optimal R_m value remains effective across different resolutions and parameter configurations.

Extensive numerical experiments revealed important properties of the optimal R_m:

- R_{m0} is largely independent of the number of collocation points
- R_{m0} generally decreases with increasing network depth
- Multi-R_m-ELM consistently outperforms Single-R_m-ELM in accuracy
- Using optimal R_m values, ELM dramatically outperforms both classical and high-order FEM for the same computational cost

The systematic determination of optimal hyperparameters represents a significant advancement in making ELM-based methods practical and effective for solving real-world PDE problems. By removing the need for manual tuning of network parameters, these algorithms enhance the reliability and accessibility of neural network PDE solvers.

7 Advanced Architectural Innovations

As randomized neural network methods continue to evolve, researchers have introduced advanced architectural innovations to address limitations of standard ELM approaches and further improve performance for challenging PDE problems.

7.1 Hidden-Layer Concatenated ELM (HLConcELM)

A significant advancement in ELM architecture is the Hidden-Layer Concatenated ELM (HLConcELM) approach [Ni and Dong, 2022], which overcomes a fundamental limitation of conventional ELM methods. Traditional ELM requires the last hidden layer to be wide to achieve high accuracy, with narrow last hidden layers invariably producing poor results regardless of other network configuration aspects.

HLConcELM introduces a modified feedforward neural network architecture where all hidden nodes across all hidden layers are directly connected to the output layer through a logical concatenation operation. The output of HLConcELM is expressed as:

$$u(x) = \sum_{i=1}^{L-1} \sum_{j=1}^{M_i} \omega_{ij} \eta_j^{(i)}(x) = \sum_{i=1}^{L-1} \boldsymbol{\eta}^{(i)}(x) \boldsymbol{\omega}_i^T = \boldsymbol{\eta}(x) \boldsymbol{\omega}^T \tag{31}$$

where $\eta_j^{(i)}(x)$ is the output field of the j-th node in the i-th hidden layer, and ω_{ij} is the weight coefficient connecting this hidden node to the output node. The logical concatenation is represented by $\boldsymbol{\eta} = [\boldsymbol{\eta}^{(1)}, \boldsymbol{\eta}^{(2)}, ..., \boldsymbol{\eta}^{(L-1)}]$.

This architecture provides several important theoretical advantages:

1. **Non-decreasing representation capacity**: Adding network complexity (either through wider or deeper networks) never reduces the representation capacity and typically enhances it

2. **Broader basis function set**: By utilizing outputs from all hidden layers, HLConcELM creates a richer set of basis functions for representing solutions

3. **Architectural flexibility**: HLConcELM achieves high accuracy regardless of whether the last hidden layer is narrow or wide

Numerical experiments across a wide range of PDEs demonstrate HLConcELM's superior performance:

- For networks with narrow last hidden layers (e.g., [2,800,50,1]), where conventional ELM completely fails (errors $\approx 10^1$ to 10^2), HLConcELM achieves high accuracy (errors $\approx 10^{-6}$ to 10^{-8})

- HLConcELM maintains exponential convergence with respect to both the number of collocation points and network width

- The method performs effectively with both shallow and deep network architectures, even with multiple narrow hidden layers

- HLConcELM can be combined with domain decomposition and block time-marching for complex problems like the viscous Burgers' equation and Korteweg-de Vries equation

This innovation addresses a fundamental limitation in conventional ELM methodology, providing greater architectural flexibility and reliability for neural network-based PDE solvers. The ability to effectively utilize all hidden nodes across all hidden layers results in a more powerful and versatile approach for solving both linear and nonlinear PDEs.

8 Randomized Neural Networks for Inverse Problems

Beyond forward PDE solving, randomized neural networks have also shown exceptional promise in addressing inverse problems, where unknown parameters and solution fields must be determined simultaneously based on sparse and potentially noisy measurement data.

8.1 Random-Weight Neural Networks for Inverse Parametric PDEs

The extension of randomized neural networks to inverse parametric PDEs [Dong and Wang, 2023] represents a significant advancement in this challenging domain. The method addresses inverse problems of the form:

$$\alpha_1 L_1(u) + \alpha_2 L_2(u) + \cdots + \alpha_n L_n(u) + F(u) = f(x), \quad x \in \Omega \tag{32}$$

$$Bu(x) = g(x), \quad x \in \partial\Omega \tag{33}$$

$$Mu(\xi) = S(\xi), \quad \xi \in \Omega_s \subset \Omega \tag{34}$$

where α_i are unknown parameters (constants or field distributions) to be determined along with the solution field $u(x)$ based on boundary conditions and sparse measurement data $S(\xi)$.

The approach employs domain decomposition with multiple local randomized neural networks and introduces three specialized algorithms:

1. **NLLSQ**: A nonlinear least squares algorithm with perturbations that determines inverse parameters and network weights simultaneously
2. **VarPro-F1**: A variable projection formulation that eliminates inverse parameters to create a reduced problem about network weights only
3. **VarPro-F2**: A variable projection formulation that eliminates network weights to create a reduced problem about inverse parameters only

For smooth solutions with noise-free data, the method demonstrates exponential convergence with respect to the number of collocation points and training parameters. With sufficient resolution, errors can reach levels close to machine precision, with the solution field accurate to 10^{-8} and inverse parameters accurate to 10^{-9} or better.

A remarkable feature of the approach is its robustness to noise in measurement data. For problems with 1-10% noise, the method still achieves high accuracy in parameter identification, with relative errors typically orders of magnitude smaller than the noise level. This robustness is enhanced through techniques like measurement residual scaling and regularization of output-layer coefficients.

Numerical experiments across various inverse problems, including parametric Poisson equations, nonlinear Helmholtz equations, viscous Burgers' equations, and identification of unknown coefficient fields, demonstrate the method's versatility and effectiveness. Compared to Physics-Informed Neural Networks (PINNs) for inverse problems, the randomized approach achieves accuracy several orders of magnitude higher with significantly faster training times.

8.2 Bayesian Physics-Informed ELM for Inverse Problems with Uncertainty Quantification

The Bayesian Physics-Informed Extreme Learning Machine (BPIELM) [Liu et al., 2023] extends the randomized neural network approach to provide uncertainty quantification for inverse problems with noisy data. This method combines the computational efficiency of PIELM with Bayesian inference to quantify uncertainties in both forward and inverse PDE problems.

BPIELM integrates Bayesian principles by:

- Introducing a prior distribution on output weights: $p(x|g) = \mathcal{N}(0, g^{-1}I)$
- Defining a likelihood function based on measurement noise: $p(Y|x, \sigma^2) = \frac{1}{(2\pi\sigma^2)^{N_c/2}} \exp\left(-\frac{\|Y-Hx\|^2}{2\sigma^2}\right)$
- Computing a posterior distribution for output weights
- Optimizing hyperparameters using the Evidence Procedure

This Bayesian framework provides not only point estimates of the solution and unknown parameters but also quantifies the uncertainty in these

estimates. The uncertainty estimates correlate strongly with actual prediction errors, providing valuable reliability information for inverse problem solutions.

For inverse problems with noisy data, BPIELM demonstrates several advantages:

- More accurate parameter identification compared to deterministic PIELM
- Meaningful confidence intervals that correctly capture the true parameter values
- Robustness to the number of hidden neurons
- Ability to identify multiple unknown parameters simultaneously

Together, these advances in randomized neural network methods for inverse problems offer powerful tools for parameter identification, coefficient field reconstruction, and uncertainty quantification in various scientific and engineering applications.

9 Specialized Methods for Interface Problems

Interface problems represent a particularly challenging class of PDEs that arise in multiphase flow, fluid-structure interactions, and other multiphysics applications where different physical phenomena couple across interfaces with potentially complex geometries. Traditional numerical methods often face significant challenges with these problems, especially when interfaces have intricate shapes or move over time.

9.1 Local Randomized Neural Networks for Interface Problems

The Local Randomized Neural Networks (LRNNs) method [Li et al., 2022] provides an elegant approach to solving interface problems by combining domain decomposition with randomized neural networks. For a domain Ω divided by an interface Γ into subdomains Ω_1 and Ω_2, the elliptic interface problem takes the form:

$$-\nabla \cdot (\beta(x)\nabla u) = f, \qquad x \in \Omega, \tag{35}$$

$$[u] = g_1, \qquad x \in \Gamma, \tag{36}$$

$$[\beta(x)\nabla u \cdot n] = g_2, \qquad x \in \Gamma, \tag{37}$$

$$u = g_D, \qquad x \in \partial\Omega \tag{38}$$

where $\beta(x)$ is a piecewise positive constant function, $[u]$ denotes the jump of u across the interface, and n is the unit outward normal vector on Γ.

The core idea of LRNNs is to use separate randomized neural networks to approximate the solution in each subdomain:

$$u_\rho^1 = \sum_{k=1}^{m} \alpha_k^1 \psi_k^1, \quad u_\rho^2 = \sum_{k=1}^{m} \alpha_k^2 \psi_k^2 \tag{39}$$

These approximations are combined with interface and boundary conditions to form a linear system that can be solved using least squares methods. To handle the relative importance of different conditions, weighting factors are introduced:

$$AX = F, \tag{40}$$
$$\gamma_1 BX = \gamma_1 G_1, \tag{41}$$
$$\gamma_2 CX = \gamma_2 G_2, \tag{42}$$
$$\gamma_3 DX = \gamma_3 G_D \tag{43}$$

For problems with time-dependent interfaces, the method employs a space-time approach where time and space variables are treated equally. The interface becomes a surface in the space-time domain, and two RNNs approximate the solution throughout this domain. This approach avoids time-stepping iteration and accumulation errors, solving the entire problem in one least-squares solution.

Numerical experiments demonstrate the method's effectiveness for a wide range of challenges:

- Complex geometries: Accurately handling flower-shaped interfaces in 2D and spherical interfaces in 3D
- Multiple interfaces: Successfully solving problems with three or more interfaces using multiple RNNs
- High-dimensional problems: Scaling to 20 dimensions with hyperplane interfaces
- Large coefficient variations: Maintaining accuracy even with diffusion coefficient ratios up to 10^{12}
- Dynamic interfaces: Effectively handling time-dependent problems with moving interfaces

The LRNNs method offers significant advantages for interface problems:

1. Mesh-free approach eliminates complex interface meshing requirements
2. Avoids expensive optimization processes required by deep neural networks
3. Achieves spectral-like accuracy for smooth solutions
4. Handles large variations in material properties across interfaces
5. Space-time approach eliminates temporal error accumulation
6. Shows good scalability to higher dimensions

These characteristics make LRNNs particularly suitable for multiphysics applications where interfaces have complex geometries or evolve over time, providing an efficient alternative to both traditional mesh-based methods and optimization-heavy deep learning approaches.

10 Extreme Learning Machine Approaches

10.1 ELM-FBPINN: Linearizing Physics-Informed Neural Networks

Extreme Learning Machines (ELMs) represent a crucial advancement in neural network training, where only the output layer weights are trained while inner layer parameters remain fixed after random initialization. The ELM-FBPINN (Extreme Learning Machine Finite Basis Physics-Informed Neural Network) method [Anderson et al., 2023] leverages this approach to significantly accelerate the training of physics-informed neural networks.

The key insight of ELM-FBPINN is transforming the non-convex optimization problem in traditional PINNs into a linear least-squares problem:

$$\min_{\mathbf{a}\in\mathbb{R}^{CJ}} \frac{1}{2}\|D_I(M\mathbf{a}-\mathbf{c})\|_2^2 + \frac{1}{4}\|D_B(B\mathbf{a}-\mathbf{g})\|_2^2, \tag{44}$$

where $\mathbf{a}$ contains the trainable weights, and matrices M and B encode the PDE operators applied to the basis functions.

The ELM-FBPINN approach combines domain decomposition with ELMs, placing separate networks in overlapping subdomains to form a global approximation:

$$u(x, \mathbf{a}) = \sum_{j=1}^{J} \omega_j(x) \sum_{c=1}^{C} a_{j,c} \sigma(\mathbf{w}_{j,c} \cdot x + b_{j,c}), \tag{45}$$

where $\omega_j(x)$ are window functions that localize each network to its subdomain.

Numerical results on a damped harmonic oscillator problem demonstrate that ELM-FBPINN achieves comparable accuracy to fully-trained FBPINN but with training times that are orders of magnitude faster. Additionally, analysis shows that the condition number of the resulting linear system remains relatively stable as the number of subdomains increases, indicating good scalability.

10.2 Bayesian Physics-Informed ELM for Noisy Data

Building on the ELM concept, the Bayesian Physics-Informed Extreme Learning Machine (BPIELM) [Liu et al., 2023] extends the approach to handle noisy data while providing uncertainty quantification. BPIELM addresses three key limitations of standard PIELM approaches:

1. Overfitting to noisy data
2. Sensitivity to the number of hidden neurons
3. Lack of uncertainty quantification

BPIELM incorporates Bayesian principles by:

- Introducing a prior distribution on output weights: $p(x|g) = \mathcal{N}(0, g^{-1}I)$
- Defining a likelihood function based on measurement noise: $p(Y|x, \sigma^2) = \frac{1}{(2\pi\sigma^2)^{N_c/2}} \exp\left(-\frac{\|Y-Hx\|^2}{\sigma^2}\right)$
- Computing a posterior distribution: $p(x|Y, g, \sigma^2) = \frac{p(Y|x,\sigma^2)p(x|g)}{p(Y|g,\sigma^2)}$

The posterior is a Gaussian distribution with mean $\mu = \sigma^{-2}RH^TY$ and variance $R = (gI + \sigma^{-2}H^TH)^{-1}$, where hyper-parameters g and σ are optimized using the Evidence Procedure.

Numerical experiments on Poisson, advection, and diffusion equations demonstrate that BPIELM provides more accurate predictions than PIELM, particularly with increasing noise levels and fewer sensors. Additionally, BPIELM is less sensitive to the number of hidden neurons and provides meaningful uncertainty estimates that correlate with prediction errors.

10.3 Fourier-Induced PIELM for Biharmonic Equations

For high-order PDEs like the biharmonic equation, traditional activation functions in PIELM can lead to instability and limited precision. The Fourier-induced Physics-Informed Extreme Learning Machine (FPIELM) [Li et al., 2023] addresses this issue by using Fourier-type activation functions, specifically sine functions.

The biharmonic equation $\Delta^2 u(x) = f(x)$ requires computation of fourth-order derivatives, which can be problematic with sigmoid, Gaussian, or tangent activation functions due to their bounded nature and complex high-order derivative behavior.

FPIELM leverages the properties of sine functions, which:

- Generate consistent responses for both narrow and large-range inputs
- Handle data across various scales
- Exhibit predictable derivative patterns, simplifying high-order derivative computation

Numerical results across various domain types (regular domains, hexagram domains, and three-dimensional domains with holes) show that FPIELM consistently achieves higher accuracy than PIELM implementations using sigmoid, Gaussian, or tangent activation functions. Additionally, FPIELM shows more stable performance across different scale factors for weight initialization and numbers of hidden neurons.

10.4 Coupled ELM for Biharmonic Equations

The Coupled Extreme Learning Machine (CELM) [Li et al., 2022] offers an alternative approach to solving biharmonic equations. Instead of directly handling the fourth-order derivatives, CELM transforms the biharmonic equation into two coupled second-order Poisson equations by introducing an auxiliary variable $v = \Delta u$:

$$\begin{cases} \Delta v = f, & \text{in } \Omega \\ v = h, & \text{on } \partial\Omega \end{cases} \quad \text{and} \quad \begin{cases} \Delta u = v, & \text{in } \Omega \\ u = g, & \text{on } \partial\Omega \end{cases} \tag{46}$$

CELM uses two separate ELM networks to approximate the solution u and the auxiliary variable v:

$$u(x; \tilde{W}, \tilde{b}) = \sigma(x^T \cdot \tilde{W} + \tilde{b}^T) \cdot \tilde{\beta} \tag{47}$$

$$v(x; \hat{W}, \hat{b}) = \sigma(x^T \cdot \hat{W} + \hat{b}^T) \cdot \hat{\beta} \tag{48}$$

By coupling these networks together, CELM transforms the fourth-order problem into a system of two second-order problems, which are more amenable to ELM approximation.

Numerical experiments demonstrate that CELM can achieve very high accuracy (relative errors as low as 10^{-14}) when using appropriate activation functions (sine or trigonometric combinations) and initialization strategies. The method also shows excellent performance on complex geometries, including irregular 2D domains and 3D domains with holes, highlighting its mesh-free advantage compared to traditional numerical methods.

10.5 ELM-Based Methods for High-Dimensional PDEs

For high-dimensional PDEs, which are challenging due to the curse of dimensionality, the ELM method and its variant combined with the approximate Theory of Functional Connections (ELM/A-TFC) [Wang and Dong, 2024] provide effective solutions.

The ELM method for high-dimensional PDEs:

- Represents the solution using a randomized neural network where only output-layer weights are trained
- Uses random collocation points instead of grid-based points to avoid exponential complexity
- Solves the resulting system using linear or nonlinear least squares methods

The ELM/A-TFC method enhances this approach by systematically enforcing boundary conditions using a hierarchically truncated TFC expression:

$$u(x) = g(x) - \mathcal{A}_\Omega g(x) + \mathcal{A}_\Omega C(x) \tag{49}$$

where $g(x)$ is approximated by an ELM network, and $\mathcal{A}_\Omega$ is a truncated operator that retains only dominant terms to avoid exponential growth in complexity.

Both methods have demonstrated remarkable capabilities in solving high-dimensional PDEs (up to 15 dimensions) with high accuracy and computational efficiency. They achieve exponential convergence with respect to the number of training parameters and boundary collocation points and significantly outperform traditional PINNs in both accuracy and computational cost.

11 Random Feature Methods for Multiscale Problems

11.1 Randomized Radial Basis Function Neural Network

Multiscale elliptic problems present significant challenges for traditional numerical methods, requiring extremely fine meshes to capture high-frequency components. The Randomized Radial Basis Function Neural Network (RRNN) [Wu et al., 2023] offers an innovative approach to address these challenges.

RRNN builds on the Radial Basis Function Neural Network (RBFNN) framework but adopts randomization principles from Extreme Learning Machines. The output of the RRNN is given by:

$$u_N(x) = \sum_{i=1}^{J} w_i e^{-\sigma_i \|x-c_i\|^2}, \tag{50}$$

where the parameters σ_i (shape coefficients) and c_i (center coefficients) are randomly assigned and fixed, while only the weights w_i are optimized.

Key innovations in the RRNN approach include:

- **Domain Decomposition**: The computational domain is divided into non-overlapping subdomains, transforming high-frequency data into a low-frequency space within each subdomain.
- **Weak Formulation**: RRNN employs a weak formulation to transfer derivatives to test functions, avoiding difficulties related to drastic gradient changes.
- **Least Squares Solution**: The weights are determined by solving a linear least-squares problem, unlike gradient descent-based methods.

Numerical experiments on various multiscale elliptic problems (periodic coefficient cases, double-scale cases, three-scale problems) demonstrated that RRNN achieves 2-3 orders of magnitude reduction in error and significantly

faster training times compared to gradient descent-based methods. The method maintains its advantages as the scale parameter ε decreases, showing robust performance for increasingly oscillatory problems.

11.2 Runge-Kutta Random Feature Method for Dynamic Problems

The Runge-Kutta Random Feature Method (RK-RFM) [Deng and He, 2023] extends the Random Feature Method (RFM) to tackle time-dependent problems, particularly multiphase flow problems of cells. RK-RFM combines RFM for spatial discretization with explicit Runge-Kutta methods for temporal discretization.

RK-RFM employs domain decomposition with partition of unity (PoU) functions and random feature functions constructed from two-layer neural networks. The key innovation is in the coupling of the RFM approach with the explicit Runge-Kutta schemes for time evolution:

$$D_1(x) = F(x, t_k, \phi_{t_k}(x)), \tag{51}$$

$$D_2(x) = F(x, t_k + c_2\Delta t, \phi_{t_k}(x) + a_{21}\Delta t D_1(x)), \tag{52}$$

$$\vdots \tag{53}$$

$$D_p(x) = F(x, t_k + c_p\Delta t, \phi_{t_k}(x) + \Delta t \sum_{i=1}^{p-1} a_{pi} D_i(x)), \tag{54}$$

$$\phi_{t_{k+1}}(x) = \phi_{t_k}(x) + \Delta t \sum_{i=1}^{p} b_i D_i(x) \equiv \phi_{t_k}(x) + \Delta t H(x, t_k, \phi_{t_k}(x), \Delta t), \tag{55}$$

where a_{pi}, b_i, and c_i are coefficients of the Runge-Kutta method.

Rigorous theoretical error estimates are provided, showing that the error consists of two components: one from the RFM approximation (inversely proportional to Δt) and one from the RK time discretization (proportional to Δt^p). The optimal time step size is determined by balancing these two error sources.

Numerical experiments validate the theoretical findings and demonstrate the method's effectiveness in simulating complex multiphase flow problems involving hundreds of cells with stable interfaces and physically realistic interactions.

11.3 Transferable Neural Networks for PDEs

Transfer learning approaches for PDEs typically involve pre-training neural networks using either the PDE's formulation or solution data, limiting their transferability across different PDEs. The Transferable Neural Network (TransNet) [Zhang et al., 2023] approach introduces a novel pre-training strategy that constructs a neural feature space without using any specific PDE information.

The TransNet approach consists of three main components:

1. **Re-parameterization of Hidden Neurons**: Each hidden neuron $\sigma(w_m^T y + b_m)$ is re-parameterized to separate location and shape parameters:
$$\sigma(w_m^T y + b_m) = \sigma(\gamma_m(a_m^T y + r_m)) \tag{56}$$

2. **Generating Uniformly Distributed Neurons**: Location parameters are generated to ensure uniform distribution across the domain.

3. **Tuning Shape Parameters**: Shape parameters are tuned using Gaussian random fields as auxiliary functions.

A key theoretical result establishes that the proposed method generates uniformly distributed neurons:

$$E[D_M^\tau(y)] = \tau \quad \text{for all } \|y\|_2 \leq 1 - \tau \tag{57}$$

TransNet was evaluated on various PDEs, including Poisson equations in different domains, steady-state Navier-Stokes equations, and time-dependent PDEs. Compared to baseline methods (random feature models and PINNs), TransNet achieved:

- Several orders of magnitude smaller MSE
- Significantly faster training times
- Successful transfer across different PDEs without retraining
- More uniform neuron distribution throughout the domain, ensuring equal expressive power

The independence from specific PDE information, avoidance of gradient-based training, and uniform distribution of neurons make TransNet a promising approach for solving various types of PDEs efficiently.

12 Comparison of Methods

13 Conclusion

This survey has explored the remarkable evolution of randomized neural networks for solving partial differential equations, tracing the development from foundational theories to specialized state-of-the-art methods. The journey began with seminal works on Random Vector Functional-Link networks and Extreme Learning Machines, which established the theoretical foundation that randomly assigning inner layer parameters while only training output weights could maintain universal approximation capabilities with dramatically improved computational efficiency.

Building on these foundations, physics-informed randomized methods emerged as transformative approaches for PDE solving. By integrating physical constraints with randomized networks, methods like PIELM transformed the challenging non-convex optimization problem of traditional PINNs into linear least-squares problems that can be solved analytically, achieving orders of magnitude faster training while maintaining or improving solution accuracy. Further refinements led to specialized approaches for high-order PDEs such as biharmonic equations, where techniques like FPIELM and CELM demonstrated the versatility of randomized networks in handling complex differential operators.

Domain decomposition strategies represent another significant advancement, with methods like locELM combining local randomized networks with interfacial continuity conditions to achieve exponential convergence rates for a wide range of PDEs. Architectural innovations such as HLConcELM further extended the capabilities of randomized networks by enabling effective utilization of all hidden nodes across all layers, providing greater flexibility and reliability.

Perhaps most impressively, specialized problem-specific methods have been developed to address particularly challenging PDE classes—RRNN for multiscale problems, RK-RFM for dynamic systems, TransNet for transferable PDE solving, and LRNNs for interface problems. These methods harness the fundamental advantages of randomization while incorporating problem-specific structures to achieve remarkable performance for their target applications.

Beyond forward PDE solving, randomized neural networks have also demonstrated exceptional effectiveness for inverse problems and uncertainty quantification, with methods like BPIELM providing not only accurate parameter identification but also meaningful confidence intervals for predic-

tions.

Across this diverse landscape of methods, several common threads emerge:

1. **Linearization of Learning**: By fixing randomly initialized inner layer parameters, these methods transform complex non-convex optimization into linear or simpler nonlinear problems, dramatically improving computational efficiency.

2. **Exponential Convergence**: Many randomized methods demonstrate exponential or near-exponential convergence with respect to degrees of freedom, providing high-accuracy solutions with relatively few parameters.

3. **Mesh-Free Flexibility**: The inherent mesh-free nature of these approaches enables handling of complex geometries and high-dimensional problems that challenge traditional mesh-based methods.

4. **Problem-Specific Adaptations**: Tailoring the randomized framework to specific PDE characteristics has led to specialized methods that significantly outperform general-purpose approaches for their target applications.

Looking forward, several exciting research directions emerge. First, deeper theoretical understanding of these methods, including rigorous error bounds and convergence analysis for nonlinear and high-dimensional cases, will strengthen their mathematical foundation. Second, adaptive methods that automatically select optimal domain decomposition, network architecture, and hyperparameters based on problem characteristics promise to enhance both performance and usability. Third, integration with traditional numerical methods may yield hybrid approaches that combine the strengths of both paradigms. Finally, extensions to increasingly complex multiphysics problems involving coupled systems, multiscale phenomena, and uncertainty quantification represent fertile ground for future advances.

The remarkable progress in randomized neural networks for PDEs over the past decade demonstrates the power of combining fundamental mathematical insights with innovative computational approaches. By transforming the way we represent and solve differential equations, these methods are not merely incremental improvements but represent a fundamental shift in computational science, with the potential to address previously intractable problems across diverse scientific and engineering domains.

References

Anderson, S., Dolean, V., Moseley, B., and Pestana, J. (2023). ELM-FBPINN: Efficient finite-basis physics-informed neural networks. *arXiv preprint*.

Deng, Y. and He, Q. (2023). Runge–Kutta random feature method for solving multiphase flow problems of cells. *arXiv preprint*.

Dong, S. and Li, Z. (2021). Local extreme learning machines and domain decomposition for solving linear and nonlinear partial differential equations. *Computer Methods in Applied Mechanics and Engineering*, 383:113894.

Dong, S. and Wang, Y. (2023). A method for computing inverse parametric PDE problems with random-weight neural networks. *arXiv preprint*.

Dong, S. and Yang, N. (2021). On computing the hyperparameter of extreme learning machines: Algorithm and application to computational PDEs. *SIAM Journal on Scientific Computing*, 43(5):A3421–A3447.

Dwivedi, V. and Srinivasan, B. (2020). Physics informed extreme learning machine (PIELM) - A rapid method for the numerical solution of partial differential equations. *Neurocomputing*, 391:96–118.

Dwivedi, V. and Srinivasan, B. (2020). Solution of biharmonic equation in complicated geometries with physics informed extreme learning machine. *Journal of Computing and Information Science in Engineering*, 20(6):061003.

Huang, G. B., Zhu, Q. Y., and Siew, C. K. (2006). Extreme learning machine: Theory and applications. *Neurocomputing*, 70(1-3):489–501.

Igelnik, B. and Pao, Y. H. (1995). Stochastic choice of basis functions in adaptive function approximation and the functional-link net. *IEEE Transactions on Neural Networks*, 6(6):1320–1329.

Li, X., Wu, J., Ding, Z., Tai, X., Liu, L., and Wang, Y. (2022). Extreme learning machine-assisted solution of biharmonic equations via its coupled schemes. *arXiv preprint*.

Li, X., Wu, J., Huang, Y., Ding, Z., Tai, X., Liu, L., and Wang, Y. (2023). Augmented physics-informed extreme learning machine to solve the biharmonic equations via Fourier expansions. *arXiv preprint*.

Li, Y., Liu, B., and Xiao, X. (2023). Extreme learning machine with kernels for solving elliptic partial differential equations. *Digital Signal Processing*, 133:103873.

Liu, X., Lin, S., Fang, J., and Xu, Z. (2015). Is extreme learning machine feasible? A theoretical assessment (part I). *IEEE Transactions on Neural Networks and Learning Systems*, 26(1):7–20.

Liu, X., Yao, W., Peng, W., and Zhou, W. (2023). Bayesian physics-informed extreme learning machine for forward and inverse PDE problems with noisy data. *arXiv preprint*.

Liu, X., Mao, T., and Xu, J. (2025). Integral representations of Sobolev spaces via ReLU^k activation function and optimal error estimates for linearized networks. *Preprint*.

Ni, Y. and Dong, S. (2022). Numerical computation of partial differential equations by hidden-layer concatenated extreme learning machine. *Journal of Scientific Computing*, 93(1):13.

Wang, Y. and Dong, S. (2024). An extreme learning machine-based method for computational PDEs in higher dimensions. *Computer Methods in Applied Mechanics and Engineering*.

Wu, Y., Liu, Z., Sun, W., and Qian, X. (2023). Randomized radial basis function neural network for solving multiscale elliptic equations. *arXiv preprint*.

Zhang, Z., Bao, F., Ju, L., and Zhang, G. (2023). Transferable neural networks for partial differential equations. *arXiv preprint*.

Li, Y. and Wang, F. (2022). Local randomized neural networks methods for interface problems. *Journal of Computational Physics*, 471:111624.

Table 1: Evolution of Randomized Neural Network Methods for PDEs

Method	Training Type	Speed	Accuracy	Specific Advan
Foundational Approaches				
RVFL (1995)	Linear Least Squares	Very Fast	Moderate	Universal Approximati
ELM (2006)	Linear Least Squares	Very Fast	Moderate	Theoretical Foundatio
Physics-Informed Methods				
PINN	Gradient Descent	Slow	Moderate	Flexibility, No Me
PIELM	Linear Least Squares	Very Fast	High	Linear PDEs, Complex
ELM-FBPINN	Linear Least Squares	Very Fast	High	Linearized Optimi
BPIELM	Bayesian Inference	Fast	High	Uncertainty Quanti
High-Order PDE Methods				
FPIELM	Linear Least Squares	Fast	High	Biharmonic Equa
CELM	Linear Least Squares	Fast	Very High	Coupled System Ap
Domain Decomposition and Architectural Innovations				
locELM	Linear/Nonlinear Least Squares	Fast	Very High	Linear/Nonlinear
HLConcELM	Linear/Nonlinear Least Squares	Fast	Very High	Architectural Flex
LRNNs	Linear Least Squares	Fast	Very High	Interface Proble
Specialized Problem-Specific Methods				
RRNN	Linear Least Squares	Fast	Very High	Multiscale Prob
RK-RFM	Linear Least Squares	Fast	High	Time-Dependent P
TransNet	Linear Least Squares	Fast	Very High	Transferability Acro
ELM/A-TFC	Linear Least Squares	Fast	High	High-Dimensional
ELM for Inverse	Linear/Nonlinear	Fast	High	Parameter Identifi